Assignment 1 — From “It Trains” to “I Know Why It’s Slow”

Part A — Build a neural net from scratch

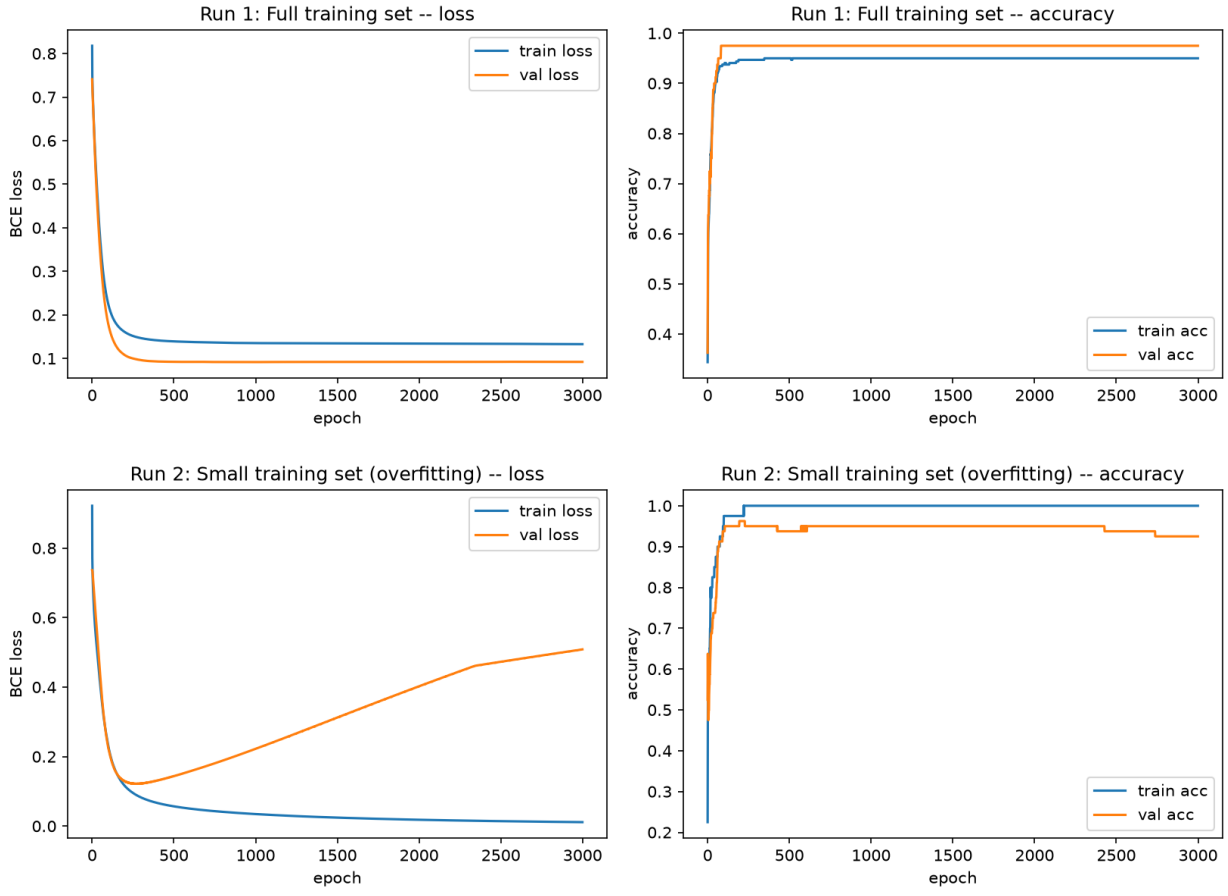
Implementation of a 2-layer MLP (2 → h → 1) in pure NumPy
‘forward’: Activation function for non-linearity is ReLU
‘loss-function': Binary cross-entropy
Final Train/Val loss and accuracy for both runs (the plateauing run and the overfitting run)

```
======================================================================
SUMMARY
======================================================================
Run                          Train Loss     Val Loss     Train Acc   Val Acc
1: Plateau (n=320)           0.1328         0.0921       0.950       0.975
2: Overfit (n=40)            0.0114         0.5085       1.000       0.925

======================================================================
RUN 1: full training set (n_train=320), watching for plateau
======================================================================
epoch    1  | train_loss 0.8178 train_acc 0.344 | val_loss 0.7409 val_acc 0.362
epoch  200  | train_loss 0.1608 train_acc 0.947 | val_loss 0.1078 val_acc 0.975
epoch  400  | train_loss 0.1415 train_acc 0.950 | val_loss 0.0930 val_acc 0.975
epoch  600  | train_loss 0.1378 train_acc 0.950 | val_loss 0.0921 val_acc 0.975
epoch  800  | train_loss 0.1360 train_acc 0.950 | val_loss 0.0917 val_acc 0.975
epoch 1000 | train_loss 0.1352 train_acc 0.950 | val_loss 0.0917 val_acc 0.975
epoch 1200 | train_loss 0.1350 train_acc 0.950 | val_loss 0.0919 val_acc 0.975
epoch 1400 | train_loss 0.1348 train_acc 0.950 | val_loss 0.0920 val_acc 0.975
epoch 1600 | train_loss 0.1346 train_acc 0.950 | val_loss 0.0921 val_acc 0.975
epoch 1800 | train_loss 0.1345 train_acc 0.950 | val_loss 0.0921 val_acc 0.975
epoch 2000 | train_loss 0.1343 train_acc 0.950 | val_loss 0.0921 val_acc 0.975
epoch 2200 | train_loss 0.1341 train_acc 0.950 | val_loss 0.0921 val_acc 0.975
epoch 2400 | train_loss 0.1339 train_acc 0.950 | val_loss 0.0921 val_acc 0.975
epoch 2600 | train_loss 0.1335 train_acc 0.950 | val_loss 0.0923 val_acc 0.975
epoch 2800 | train_loss 0.1331 train_acc 0.950 | val_loss 0.0922 val_acc 0.975
epoch 3000 | train_loss 0.1328 train_acc 0.950 | val_loss 0.0921 val_acc 0.975

======================================================================
RUN 2: small training subset (n_train=40), same val set -> overfitting
======================================================================
epoch    1  | train_loss 0.9218 train_acc 0.225 | val_loss 0.7365 val_acc 0.525
```

epoch 200 | train_loss 0.1161 train_acc 0.975 | val_loss 0.1292 val_acc 0.963
epoch 400 | train_loss 0.0665 train_acc 1.000 | val_loss 0.1314 val_acc 0.950
epoch 600 | train_loss 0.0502 train_acc 1.000 | val_loss 0.1578 val_acc 0.938
epoch 800 | train_loss 0.0410 train_acc 1.000 | val_loss 0.1885 val_acc 0.950
epoch 1000 | train_loss 0.0346 train_acc 1.000 | val_loss 0.2222 val_acc 0.950
epoch 1200 | train_loss 0.0298 train_acc 1.000 | val_loss 0.2577 val_acc 0.950
epoch 1400 | train_loss 0.0260 train_acc 1.000 | val_loss 0.2934 val_acc 0.950
epoch 1600 | train_loss 0.0229 train_acc 1.000 | val_loss 0.3303 val_acc 0.950
epoch 1800 | train_loss 0.0204 train_acc 1.000 | val_loss 0.3665 val_acc 0.950
epoch 2000 | train_loss 0.0182 train_acc 1.000 | val_loss 0.4018 val_acc 0.950
epoch 2200 | train_loss 0.0164 train_acc 1.000 | val_loss 0.4370 val_acc 0.950
epoch 2400 | train_loss 0.0149 train_acc 1.000 | val_loss 0.4655 val_acc 0.950
epoch 2600 | train_loss 0.0135 train_acc 1.000 | val_loss 0.4796 val_acc 0.938
epoch 2800 | train_loss 0.0124 train_acc 1.000 | val_loss 0.4944 val_acc 0.925
epoch 3000 | train_loss 0.0114 train_acc 1.000 | val_loss 0.5085 val_acc 0.925



Summary: The overfitting run matches the bias–variance curve because training performance improves while validation performance gets worse.

Part B — Training-cost estimator

Model : Phi-3-mini
P(Parameters) : 3.80e+09 params
T(Tokens) : 3.30e+12 tokens
Total training FLOPs : 7.524e+22 FLOPs
Optimizer memory : 60.8 GB
GPU assumed : A100, 312 TFLOP/s peak BF16,
40% utilization -> 1.248e+14 FLOP/s effective
Wall-clock, 1 GPU : 6977.8 days = 19.1 years
Wall-clock, 1,024 GPUs : 163.5 hours = 6.8 days

Summary: Out of the Compute, Memory, I/O walls, Compute wall is hit first
Justification: At ~61 GB, the optimizer state comfortably fits on a single 80GB A100 and data loading isn't the constraint at 3.3T tokens spread across the long run. However, ~7 × 10^{22} FLOPs: even at a generous 40% utilization on a top-of-line A100, one GPU alone would take 19 years

---

Part C — micro-benchmark

==========

Baseline Numbers Table

==========

Kernel	$N_{(data\ elements)}$	AI	GFLOP/s	Class
SAXPY(baseline)	20000000	0.083	1.67	memory-bound
GEMM (baseline)	16	1.333	7.06	memory-bound
GEMM (bigger N)	512	42.667	318.53	compute-bound

(Chane one thing -> increase GEMM N (16 -> 512); SAXPY size held fixed as control)

======DELTA vs THEORY======

AI : 1.33 -> 42.67 FLOP/byte (up)
GFLOP/s : 7.06 -> 318.53 (up)
(Theory predicted both AI and achieved GFLOP/s would rise as N grows. Observed result matches theory.)

Sanity Check: After running/comparing these results with examples given in course, I see the numbers are in the same ball mark. SAXPY's AI is exactly 0.083. It's reassuring that both scripts are computing FLOPs and bytes-moved the same way. Same for GEMM: example script gives AI=166.67 at N=2000, my assignment script gives 42.67 at N=512 — both equal N/12 exactly, as the formula predicts.

Synthesis:

‘mlp_from_scratch.py’ trains a small 2-layer MLP using matrix multiplications, while ‘training_cost_estimator.py’ shows how training compute scales dramatically with model parameters and tokens (≈6PT FLOPs). ‘roofline-bench.py’ adds the hardware perspective: operations with low arithmetic intensity are memory-bound, while higher-AI GEMMs tend toward compute-bound execution.

For actually training the larger model, my first optimization step would be to profile the dominant matrix-multiplication/GEMM workloads and determine whether they are memory-bound or compute-bound before changing the model or hardware. I would first collect per-operation runtime, FLOPs, memory traffic, arithmetic intensity, achieved GFLOP/s, and GPU utilization, then optimize the actual bottleneck rather than guessing.